

AI Audit Log

MSSV: 23127125

Họ và tên: Nguyễn Hiếu Thuận

Bài tập: HW02 - Domain Testing

FR-02: Đăng nhập & Khóa tài khoản

[Prompt 1]

- Thời gian:** 14:30 09/06/2026
- Tool:** Gemini 3.1 Pro
- Prompt:**

Đóng vai trò là một Software Tester chuyên nghiệp. Tôi đang cần áp dụng kỹ thuật Domain Testing cho chức năng FR-02: Đăng nhập và Khóa tài khoản.

Dưới đây là đặc tả (business rules) của chức năng:

- Người dùng nhập Email và Mật khẩu.
- Trường email phải dùng type="email" (có validate HTML5 format).
- Sau mỗi lần đăng nhập sai, hệ thống tăng bộ đếm lên đúng 1 đơn vị.
- Nếu đăng nhập sai từ 3 lần trở lên liên tiếp, tài khoản bị tạm khóa 30 giây (môi trường demo).
- Trả về thông báo lỗi phù hợp; không để lộ chi tiết nguyên nhân.
- Đăng nhập thành công trả về JWT Token.

Nhiệm vụ của bạn: Hãy phân tích các biến đầu vào (Email, Password) và biến trạng thái (Login failed attempts). Liệt kê chi tiết các miền giá trị (Domain) hợp lệ (Valid) và không hợp lệ (Invalid) cho từng biến. Xin đừng vội tạo Test Case, chỉ thực hiện bước phân tích Domain. Trình bày bằng Markdown rõ ràng.

- AI Output:**

Phân tích Miền giá trị (Domain Analysis) - FR-02

1. Biến đầu vào: Email

Dựa trên quy định bắt buộc sử dụng type="email" (HTML5 format) phía frontend và logic xác thực tài khoản phía backend.

Miền hợp lệ (Valid Domains):

Chuỗi ký tự tuân thủ đúng định dạng chuẩn của email HTML5 (Ví dụ: user@example.com, firstname.lastname@sub.domain.org).

Email hợp lệ VÀ đã được đăng ký (tồn tại trong cơ sở dữ liệu của hệ thống).

Miền không hợp lệ (Invalid Domains):

Trống (Empty): Không nhập gì vào trường email.

Sai định dạng (Invalid HTML5 Format): Chuỗi ký tự vi phạm cấu trúc email chuẩn (Ví dụ: userexample.com - thiếu @, @example.com - thiếu tên hộp thư, user@ - thiếu tên miền, chứa khoảng trắng hoặc ký tự đặc biệt không cho phép).

Chưa đăng ký (Unregistered): Email đúng định dạng HTML5 nhưng không tồn tại trong hệ thống.

2. Biến đầu vào: Mật khẩu (Password)

Biến này được đánh giá dựa trên sự đối chiếu với dữ liệu đã lưu trữ của Email tương ứng.

Miền hợp lệ (Valid Domains):

Chuỗi ký tự khớp chính xác (có phân biệt chữ hoa/chữ thường) với mật khẩu đã lưu của Email hợp lệ vừa nhập.

Miền không hợp lệ (Invalid Domains):

Trống (Empty): Không nhập gì vào trường mật khẩu.

Không khớp (Mismatch): Chuỗi ký tự nhập vào không đúng với mật khẩu của Email tương ứng.

3. Biến trạng thái: Số lần đăng nhập sai (Login Failed Attempts)

Đây là biến ẩn (state variable) cực kỳ quan trọng, quyết định luồng xử lý của hệ thống đối với tính năng tạm khóa tài khoản. Biến này phụ thuộc vào 2 yếu tố: Bộ đếm (Counter) và Thời gian chờ (Timeout).

Miền hợp lệ (Active State - Cho phép thực hiện đăng nhập):

Attempts = 0: Trạng thái ban đầu (người dùng mới) hoặc bộ đếm vừa được reset sau một lần đăng nhập thành công.

Attempts = 1 hoặc Attempts = 2: Người dùng đã nhập sai thông tin 1 hoặc 2 lần liên tiếp. Hệ thống ghi nhận lỗi nhưng tài khoản vẫn ở trạng thái hoạt động (Active).

Attempts >= 3 VÀ Thời gian kể từ lần sai cuối cùng >= 30 giây: Tài khoản đã vượt qua thời gian phạt. Trạng thái được mở khóa (Unlocked), cho phép người dùng tiếp tục gửi request đăng nhập.

Miền không hợp lệ (Locked State - Từ chối đăng nhập):

Attempts >= 3 VÀ Thời gian kể từ lần sai cuối cùng < 30 giây: Tài khoản đang trong thời gian bị tạm khóa. Trong miền trạng thái này, mọi thao tác submit form đăng nhập (cho dù nhập đúng hay sai Email/Password) đều phải bị hệ thống từ chối xử lý và trả về thông báo lỗi chung.

[Prompt 2]

- **Thời gian:** 21:30 09/06/2026
- **Tool:** Gemini 3.1 Pro
- **Prompt:**

Dựa vào bảng phân tích miền giá trị (Valid/Invalid Domains) và các miền kết quả kỳ vọng (01, 02, 03, 04) mà bạn vừa liệt kê, bây giờ hãy thiết kế một tập hợp các Test Case chi tiết cho kỹ thuật Domain Testing của chức năng FR-02.

Yêu cầu:

Trình bày dưới dạng bảng (Markdown Table).

Các cột bắt buộc: Test Case ID, Mục đích (Test Objective), Điều kiện tiên quyết (Precondition), Đầu vào (Email, Password), Trạng thái hệ thống (Failed Attempts), và Kết quả mong đợi (Expected Result).

Hãy kết hợp các miền giá trị sao cho bao phủ được cả 4 kịch bản kết quả (01, 02, 03, 04) mà không bị dư thừa (tránh tổ hợp bùng nổ không cần thiết).

Đảm bảo test case có tính thực tế (ví dụ: cung cấp data mẫu cho Email/Password trong bảng).

Xuất ra kết quả dưới dạng markdown mà tôi có thể thêm vào lập tức ở Main Report

- **AI Output:**

Thiết kế Test Case (Domain Testing) Dựa trên các miền giá trị đã phân tích, dưới đây là tập hợp các Test Case đại diện nhằm đảm bảo bao phủ toàn bộ các kịch bản kết quả (01, 02, 03, 04) mà không gây ra bùng nổ tổ hợp: Test Case ID Mục đích (Test Objective) Điều kiện tiền quyết (Precondition) Đầu vào (Email & Password) Trạng thái hệ thống (Failed Attempts) Kết quả mong đợi (Expected Result) TC_FR02_01 Kiểm tra đăng nhập thành công ở trạng thái bình thường (Phủ: V1 + V2 + S1 -> 01) Tài khoản user@example.com đã tồn tại, pass: Abc@123 Email: user@example.com Pass: Abc@123 S1: 0 lần sai Đăng nhập thành công, trả về JWT Token, bộ đếm giữ ở 0. TC_FR02_02 Kiểm tra đăng nhập thành công sau khi hết thời gian phạt (Phủ: V1 + V2 + S3 -> 01) Tài khoản user@example.com đã tồn tại, pass: Abc@123 Email: user@example.com Pass: Abc@123 S3: 3 lần sai, đã chờ > 30s Đăng nhập thành công, trả về JWT Token, reset bộ đếm về 0. TC_FR02_03 Kiểm tra validation UI khi bỏ trống Email (Phủ: I1 -> 02) Hệ thống đang ở trang Đăng nhập Email: (bỏ trống) Pass: Abc@123 S1: 0 lần sai Frontend chặn submit, hiển thị lỗi yêu cầu nhập Email. TC_FR02_04 Kiểm tra validation UI khi Email sai định dạng (Phủ: I2 -> 02) Hệ thống đang ở trang Đăng nhập Email: user@example.com Pass: Abc@123 S1: 0 lần sai Frontend chặn submit, hiển thị lỗi định dạng HTML5 email. TC_FR02_05 Kiểm tra validation UI khi bỏ trống Password (Phủ: I4 -> 02) Hệ thống đang ở trang Đăng nhập Email: user@example.com Pass: (bỏ trống) S1: 0 lần sai Frontend chặn submit, hiển thị lỗi yêu cầu nhập Mật khẩu. TC_FR02_06 Kiểm tra lỗi bảo mật khi Email chưa đăng ký (Phủ: I3 + S1 -> 03) Email new@example.com chưa có trong DB Email: new@example.com Pass: AnyPass123 S1: 0 lần sai Trả về lỗi chung ("Email hoặc mật khẩu không đúng"), tăng bộ đếm lên 1. TC_FR02_07 Kiểm tra lỗi bảo mật khi sai Password và đẩy lên giới hạn khóa (Phủ: V1 + I5 + S2 -> 03) Tài khoản user@example.com đã tồn tại Email: user@example.com Pass: WrongPass! S2: 2 lần sai Trả về lỗi chung, tăng bộ đếm lên 3 (Tài khoản chính thức chuyển sang trạng thái tạm khóa). TC_FR02_08 Kiểm tra từ chối đăng nhập khi tài khoản đang bị khóa, dù nhập đúng (Phủ: V1 + V2 + S4 -> 04) Tài khoản user@example.com đã tồn tại Email: user@example.com Pass: Abc@123 S4: 3 lần sai, thời gian < 30s Trả về thông báo tài khoản đang bị tạm khóa, không trả về JWT Token. TC_FR02_09 Kiểm tra từ chối đăng nhập khi tài khoản đang bị khóa, nhập sai (Phủ: I3 + I5 + S4 -> 04) Bất kỳ email/pass nào Email: fake@example.com Pass: FakePass S4: 3 lần sai, thời gian < 30s, Trả về thông báo tài khoản đang bị tạm khóa (hệ thống không check DB, ưu tiên check trạng thái lock trước).

[Prompt 3]

- **Thời gian:** 19:30 10/06/2026
- **Tool:** Gemini 3.1 Pro
- **Prompt:**

Chúng ta đã hoàn thành Domain Testing. Bây giờ, tôi muốn áp dụng kỹ thuật Boundary Value Analysis (BVA) cho chức năng FR-02. Dựa vào đặc tả, có 2 ranh giới (boundary) cần phân tích:
 Giới hạn số lần đăng nhập sai: Ngưỡng khóa tài khoản là 3 lần (n = 3).
 Giới hạn thời gian tạm khóa: Thời gian phạt là 30 giây (t = 30s).
 Nhiệm vụ của bạn:
 Bước 1 (Giải thích step-by-step): Hãy viết một đoạn ngắn giải thích cách phân tích các giá trị biên (min, max, just below, just above, on the boundary) cho 2 ranh

giới trên. Ví dụ: Với $n=3$ thì các giá trị cần test là $n=2$, $n=3$, $n=4$.
 Bước 2 (Thiết kế Test Case): Thiết kế bảng Test Case BVA chi tiết để kiểm tra các giá trị biên này. Bảng phải có các cột: Test Case ID (bắt đầu là TC_FR02_BVA_01), Mục đích, Điều kiện tiên quyết, Đầu vào & Trạng thái (Inputs & State), và Kết quả mong đợi.
 Xin hãy trình bày bằng Markdown rõ ràng để tôi có thể đưa trực tiếp vào báo cáo.

• **AI Output:**

Boundary Value Analysis (BVA)
 Bước 1: Phân tích các giá trị biên (Step-by-step Explanation)
 Để áp dụng kỹ thuật BVA cho FR-02, tôi tập trung vào 2 ranh giới quyết định việc thay đổi trạng thái của hệ thống: số lần đăng nhập sai và thời gian tạm khóa tài khoản. Theo nguyên tắc của BVA, lỗi thường tiềm ẩn ở khu vực lân cận các ranh giới này.

1. Ranh giới số lần đăng nhập sai (Ngưỡng $n = 3$):

- Just below the boundary ($n = 2$): Kiểm tra khi người dùng nhập sai lần thứ 2. Hệ thống phải ghi nhận lỗi nhưng tài khoản vẫn ở trạng thái hoạt động.
- On the boundary ($n = 3$): Kiểm tra khi người dùng nhập sai ở đúng lần thứ 3. Đây là điểm trigger kích hoạt trạng thái tạm khóa.
- Just above the boundary ($n = 4$): Kiểm tra khi người dùng tiếp tục gửi request đăng nhập lần thứ 4 (khi tài khoản vừa bị khóa). Hệ thống phải lập tức từ chối mà không cần kiểm tra DB.

2. Ranh giới thời gian tạm khóa (Ngưỡng $t = 30s$):

- Just below the boundary ($t = 29s$): Cố gắng thực hiện đăng nhập khi thời gian phạt chưa kết thúc (còn thiếu 1 giây). Hệ thống vẫn phải hiển thị lỗi bị khóa.
- On the boundary ($t = 30s$): Cố gắng thực hiện đăng nhập ngay tại thời điểm mốc phạt vừa chạm tới. Hệ thống phải cho phép xác thực (mở khóa tài khoản).
- Just above the boundary ($t = 31s$): Cố gắng đăng nhập khi thời gian phạt đã qua mức an toàn. Hệ thống phải tiếp nhận request bình thường.

Bước 2: Thiết kế Test Case (BVA Testing)
 Dưới đây là các Test Case chi tiết để kiểm thử các giá trị biên đã phân tích:

Test Case ID	Mục đích (Test Objective)	Điều kiện tiên quyết (Precondition)	Đầu vào & Trạng thái (Inputs & State)	Kết quả mong đợi (Expected Result)
TC_FR02_BVA_01	Kiểm tra biên dưới số lần sai ($n = 2$)	Tài khoản hợp lệ, bộ đếm đang là 1	Email: user@example.com Password: WrongPass State: 1 lần sai	Hệ thống báo lỗi thông tin không đúng, tăng bộ đếm lên 2, tài khoản không bị khóa.
TC_FR02_BVA_02	Kiểm tra tại biên số lần sai ($n = 3$)	Tài khoản hợp lệ, bộ đếm đang là 2	Email: user@example.com Password: WrongPass State: 2 lần sai	Hệ thống báo lỗi tài khoản bị tạm khóa 30s, tăng bộ đếm lên 3.
TC_FR02_BVA_03	Kiểm tra vượt biên số lần sai ($n = 4$)	Tài khoản vừa bị khóa, bộ đếm đang là 3	Email: user@example.com Password: AnyPass State: 3 lần sai	Hệ thống từ chối đăng nhập ngay lập tức, báo lỗi tài khoản đang bị tạm khóa.
TC_FR02_BVA_04	Kiểm tra biên dưới thời gian khóa ($t = 29s$)	Tài khoản đang bị khóa, vừa trôi qua 29 giây	Email: user@example.com Password: CorrectPass State: 3 lần sai, $t = 29s$	Đăng nhập thất bại, hệ thống báo lỗi tài khoản vẫn đang bị khóa.
TC_FR02_BVA_05	Kiểm tra tại biên thời gian khóa ($t = 30s$)	Tài khoản bị khóa, vừa trôi qua đúng 30 giây	Email: user@example.com Password: CorrectPass State: 3 lần sai, $t = 30s$	Đăng nhập thành công, trả về JWT Token, reset bộ đếm số lần sai về 0.
TC_FR02_BVA_06	Kiểm tra vượt biên thời gian khóa ($t = 31s$)	Tài khoản bị khóa, đã trôi qua 31 giây	Email: user@example.com Password: CorrectPass State: 3 lần sai, $t = 31s$	Đăng nhập thành công, trả về JWT Token, reset bộ đếm số lần sai về 0.

- **Thời gian:** 10:00 15/06/2026
- **Tool:** Gemini 3.1 Pro
- **Prompt:**

vay hay thiet ke lai file Bug_report cho phu hop nhe, luu y tra ve format markdown de toi copy vao

- **AI Output:**

```
# Bug Report
**MSSV:** 23127125 </br>
**Họ và tên:** Nguyễn Hiếu Thuận </br>
**Bài tập:** HW02 - Domain Testing </br>
**Chức năng:** FR-02 (Đăng nhập & Khóa tài khoản)
---
```

Danh sách Lỗi (Bugs) phát hiện được trên SUT

1. Bug 1: Sai logic khóa tài khoản (Khóa ở lần 2 thay vì lần 3)

* **Mô tả:** Theo đặc tả, tài khoản chỉ bị tạm khóa khi nhập sai từ 3 lần trở lên liên tiếp. Tuy nhiên, thực tế hệ thống đã khóa tài khoản ngay ở lần nhập sai thứ 2.

* **Test Case phát hiện:** `TC\_FR-02\_07`, `TC\_FR-02\_BVA\_01`

* **GitHub Issue:** [Link Issue #1] *(Bạn hãy thay link thật vào đây)*

* **Ảnh minh chứng:**

![Bug 1](images/bug1.png)

2. Bug 2: Không hiển thị thông báo "Tài khoản bị tạm khóa" (UI/UX)

* **Mô tả:** Khi tài khoản rơi vào trạng thái khóa, hệ thống không trả về thông báo lỗi chi tiết như yêu cầu ("Tài khoản bị tạm khóa 30s") mà chỉ hiển thị thông báo chung chung "Đăng nhập thất bại, vui lòng kiểm tra lại". Điều này gây hoang mang cho người dùng.

* **Test Case phát hiện:** `TC\_FR-02\_08`, `TC\_FR-02\_09`, `TC\_FR-02\_BVA\_03`

* **GitHub Issue:** [Link Issue #2] *(Bạn hãy thay link thật vào đây)*

* **Ảnh minh chứng:**

![Bug 2](images/bug2.png)

3. Bug 3: Lỗi đồng bộ trạng thái Frontend (Phải F5 mới đăng nhập lại được)

* **Mô tả:** Sau khi hết thời gian phạt 30 giây, nếu người dùng tiếp tục nhấn nút Đăng nhập thì vẫn không thành công. Người dùng bắt buộc phải F5 (tải lại toàn bộ trang) thì mới có thể đăng nhập lại bình thường. Frontend không tự động reset state timeout.

* **Test Case phát hiện:** `TC\_FR-02\_02`, `TC\_FR-02\_BVA\_05`, `TC\_FR-02\_BVA\_06`

* **GitHub Issue:** [Link Issue #3] *(Bạn hãy thay link thật vào đây)*

* **Ảnh minh chứng:**

![Bug 3](images/bug3-1.png)

4. Bug 4: Thiếu Validation chuẩn HTML5 ở trường Email

* **Mô tả:** Ô nhập Email ở Frontend đang thiếu ràng buộc validate theo định dạng email chuẩn (ví dụ: thiếu ký tự `@` nhưng vẫn submit được). Form vẫn đẩy request sai định dạng xuống tận Backend xử lý thay vì chặn ngay tại UI.

* **Test Case phát hiện:** `TC\_FR-02\_04`

* **GitHub Issue:** [Link Issue #4] *(Bạn hãy thay link thật vào đây)*

* **Ảnh minh chứng:**
![Bug 4](images/bug4.png)

FR-08: Thanh toán (Checkout)

[Prompt 1]

- **Thời gian:** 9:30 16/06/2026
- **Tool:** Gemini 3.1 Pro
- **Prompt:**

Đóng vai trò là một Software Tester chuyên nghiệp. Tôi cần áp dụng kỹ thuật Domain Testing cho chức năng FR-08: Thanh toán (Checkout).

Mục tiêu của tôi là tạo ra một bộ Test Case bao phủ tối đa các kịch bản ngoại lệ, đặc biệt là các lỗi bảo mật (tampering) để phát hiện càng nhiều bug càng tốt. Dưới đây là đặc tả hệ thống:

Chỉ người dùng đã đăng nhập mới tiến hành thanh toán được.

Tổng tiền thanh toán được tính tự động từ giỏ hàng và không cho phép người dùng chỉnh sửa trực tiếp trên UI.

Giao diện hiển thị đầy đủ danh sách sản phẩm đặt mua.

Backend phải tự tính lại tổng tiền; không chấp nhận giá trị total_amount do client gửi lên.

Sau thanh toán thành công, giỏ hàng được xóa.

Nhiệm vụ của bạn gồm 2 phần, hãy trình bày hoàn toàn bằng Markdown để tôi đưa vào báo cáo:

Phần 1: Phân tích Domain (Step-by-step Explanation)

Hãy phân tích rõ các miền giá trị (Hợp lệ / Không hợp lệ) cho các biến ẩn và biến trạng thái sau:

Biến trạng thái đăng nhập (Auth Token).

Biến trạng thái Giỏ hàng (Số lượng món, Tồn kho sản phẩm trước khi checkout).

Biến đầu vào Payload API (Giá trị total_amount client gửi lên so với thực tế).

Phần 2: Thiết kế Bảng Test Case

Từ phân tích trên, hãy sinh ra một bộ Test Case chi tiết. Kết hợp các miền giá trị để tạo ra cả Happy Path và rất nhiều Negative/Edge Cases (Ví dụ: Dùng Postman/Burp Suite để sửa total_amount thành 0, số âm; Giỏ hàng trống nhưng vẫn gọi API; Sản phẩm bị xóa/hết hàng ngay lúc bấm thanh toán; Token hết hạn...).

Bảng gồm các cột: Test Case ID (Bắt đầu là TC_FR-08_01), Tên (Test Objective), Điều kiện tiên quyết, Đầu vào & Trạng thái, Kết quả mong đợi.

- **AI Output:**

FR-08: Thanh toán (Checkout)Domain TestingĐể thực hiện Domain Testing cho FR-08, tôi không chỉ kiểm tra các thao tác trên giao diện UI (nơi người dùng bị giới hạn) mà tập trung phân tích các biến trạng thái ẩn và dữ liệu Payload API gửi từ Client lên Backend. Mục tiêu là phát hiện các lỗi hỏng bảo mật (Data Tampering) và lỗi logic đồng bộ (Race Condition). Phần 1: Phân tích Miền giá trị (Domain Analysis)Chức năng Thanh toán phụ thuộc vào 3 nhóm biến trạng thái và đầu vào cốt lõi:1. Biến trạng thái đăng nhập: Auth Token (Header Authorization)Trạng thái này

quyết định quyền truy cập vào API thanh toán. Miền hợp lệ (Valid Domains): V1: Token tồn tại, chuẩn định dạng JWT, đúng chữ ký (Signature) và còn thời hạn sử dụng. Miền không hợp lệ (Invalid Domains): I1 (Missing): Không gửi kèm Token trong Header. I2 (Expired): Token hợp lệ nhưng đã hết hạn (Expired). I3 (Tampered/Invalid): Token bị chỉnh sửa nội dung (sai chữ ký) hoặc là token fake.

2. Biến trạng thái Giỏ hàng (Cart State & Inventory) Trạng thái này kiểm tra tính hợp lệ của dữ liệu đầu vào trước khi lập đơn hàng. Miền hợp lệ (Valid Domains): V2: Giỏ hàng có ít nhất 1 sản phẩm ($\$N \geq 1\$$) VÀ Số lượng đặt mua của từng món $\leq$ Số lượng tồn kho thực tế trong DB. Miền không hợp lệ (Invalid Domains): I4 (Empty Cart): Giỏ hàng trống ($\$N = 0\$$) nhưng Client vẫn cố tình gọi API Checkout. I5 (Out of Stock): Tồn kho hiện tại trong DB $<$ Số lượng khách đặt mua (Thường xảy ra do Race Condition: khách A thêm vào giỏ, khách B mua hết, sau đó khách A mới bấm thanh toán). I6 (Deleted Product): Sản phẩm trong giỏ đã bị Admin xóa hoặc vô hiệu hóa khỏi hệ thống trước khi thanh toán.

3. Biến đầu vào Payload API: total_amount (Client vs Backend) Theo đặc tả, Backend KHÔNG ĐƯỢC tin tưởng giá trị total_amount từ Client gửi lên mà phải tự tính toán lại. Miền hợp lệ (Valid Domains): V3: Giá trị total_amount Client gửi khớp 100% với tổng tiền Backend tự tính dựa trên DB. (Hoặc Payload không gửi trường này, để Backend tự lo). Miền không hợp lệ (Invalid Domains / Tampering): I7 (Zero/Negative): Cố tình sửa payload total_amount = 0 hoặc số âm (VD: -50000). I8 (Fractional/Low): Cố tình sửa total_amount thành một số rất nhỏ (VD: 1 VNĐ). I9 (Overpriced): Cố tình sửa total_amount thành số lớn hơn thực tế (kiểm tra xem Backend có validate hay ghi nhận sai doanh thu không).

Phần 2: Thiết kế Test Case (Domain Testing) Dựa trên phân tích trên, dưới đây là bộ Test Case kết hợp kỹ thuật chặn bắt Request (Intercept) qua Postman/Burp Suite để kiểm tra độ tin cậy của API: Test Case ID Tên (Test Objective) Điều kiện tiền quyết (Precondition) Đầu vào & Trạng thái (Inputs & State) Kết quả mong đợi (Expected Result)

TC_FR-08_01 Kiểm tra thanh toán thành công (Happy Path) (Phủ: V1 + V2 + V3) User đã đăng nhập, giỏ hàng có 2 SP (Tổng 500k), tồn kho đủ. Auth: JWT hợp lệ. Cart: 2 SP hợp lệ. Payload: total_amount = 500.000 Thanh toán thành công. Backend tính đúng 500k. Trả về mã đơn hàng. Sau đó giỏ hàng tự động bị xóa (trống).

TC_FR-08_02 Kiểm tra bảo mật: Sửa tổng tiền thành 0 hoặc 1 VNĐ (Phủ: V1 + V2 + I7/I8) User đã đăng nhập, giỏ hàng có SP trị giá 500k. Dùng Postman gửi request. Payload: total_amount = 1 (hoặc 0) Backend bỏ qua giá trị 1đ của Client, tự động tính lại tổng tiền là 500k để lập đơn hàng (HOẶC trả về lỗi 400 Data Mismatch). Không được tạo đơn giá 1đ.

TC_FR-08_03 Kiểm tra bảo mật: Sửa tổng tiền thành số âm (Phủ: V1 + V2 + I7) User đã đăng nhập, giỏ hàng có SP trị giá 500k. Dùng Postman gửi request. Payload: total_amount = -50000 Backend từ chối xử lý, báo lỗi 400 Bad Request (Invalid Amount), HOẶC tự động sửa lại thành 500k đúng giá trị thực.

TC_FR-08_04 Kiểm tra bảo mật API: Giỏ hàng trống nhưng vẫn gọi API Checkout (Phủ: V1 + I4) User đã đăng nhập. Giỏ hàng hiện tại đang trống (0 SP). Dùng Postman gọi thẳng API Checkout. Cart: $\$N = 0\$$ Backend chặn lại, trả về lỗi 400 (Giỏ hàng trống, không thể lập đơn). Không tạo ra đơn hàng rỗng.

TC_FR-08_05 Kiểm tra Race Condition: Hết hàng ngay lúc bấm thanh toán (Phủ: V1 + I5) User có SP A trong giỏ. (Admin vừa vào DB set tồn kho SP A = 0). Bấm nút Checkout trên UI. Cart: Số lượng đặt > Tồn kho Thanh toán thất bại. Trả về thông báo lỗi: "Sản phẩm A đã hết hàng hoặc không đủ số lượng".

TC_FR-08_06 Kiểm tra Race Condition: Sản phẩm bị xóa trước khi thanh toán (Phủ: V1 + I6) User có SP B trong giỏ. (Admin vừa vào DB xóa SP B). Bấm nút Checkout trên UI. Cart: SP không tồn tại trong DB Thanh toán thất bại. Trả về thông báo lỗi: "Sản phẩm B không còn tồn tại".

TC_FR-08_07 Kiểm tra quyền truy cập: Gọi API không có Token (Phủ: I1) Giỏ hàng có SP. Dùng Postman xóa header Authorization. Backend từ chối truy cập, trả về lỗi 401 Unauthorized.

TC_FR-08_08 Kiểm tra quyền truy cập: Token đã hết hạn (Phủ: I2) Giỏ hàng có SP. User đăng nhập từ hôm qua (Token đã hết hạn). Bấm nút Checkout trên

UI.Auth: Expired JWTBackend từ chối truy cập (401). Frontend tự động redirect về trang Đăng nhập.

[Prompt 2]

- **Thời gian:** 15:00 22/06/2026
- **Tool:** Gemini 3.1 Pro
- **Prompt:**

Chúng ta đã hoàn thành Domain Testing. Bây giờ, tôi muốn áp dụng kỹ thuật Boundary Value Analysis (BVA) cho chức năng FR-08: Thanh toán (Checkout).

LƯU Ý CỰC KỲ QUAN TRỌNG (Để tránh Hallucination): > 1. Hệ thống SUT này HOÀN TOÀN KHÔNG CÓ cơ chế quản lý tồn kho (stock). Tuyệt đối không sinh ra các test case liên quan đến việc kiểm tra số lượng tồn kho hay "hết hàng".

2. Hãy sử dụng mock data này cho tất cả test case: Sản phẩm "iPhone 15 Pro Max", Giá: 30.000.000 VNĐ.

Dựa vào đặc tả và quan sát UI (không chặn độ dài input số lượng), có 2 ranh giới (boundary) cần phân tích ở API Checkout:

Giới hạn tối thiểu của Giỏ hàng (N - Số lượng sản phẩm): > - Ranh giới là $N = 1$ (Số lượng mua tối thiểu để thanh toán).

Giới hạn lưu trữ tổng tiền (Total Amount - Integer Overflow):

Ranh giới là giới hạn tối đa của kiểu dữ liệu số nguyên 32-bit (Max Int = 2.147.483.647).

Vì 1 sản phẩm giá 30.000.000 VNĐ, nếu người dùng mua số lượng cực lớn (VD: $N = 72$ chiếc thì tổng tiền là 2.160.000.000, vượt qua mốc Max Int).

Nhiệm vụ của bạn: Bước 1 (Giải thích step-by-step): Hãy viết một đoạn ngắn giải thích cách phân tích các giá trị biên (min, max, just below, just above, on the boundary) cho 2 ranh giới trên. Ví dụ: Với $N=1$ thì test $N=0$, $N=1$. Với Max Int, test giá trị mấp mé giới hạn và giá trị vượt giới hạn để xem Backend có bị crash không. Bước 2 (Thiết kế Test Case): Thiết kế bảng Test Case BVA chi tiết để kiểm tra các giá trị biên này. Bảng phải có các cột: Test Case ID (bắt đầu là TC_FR-08_BVA_01), Tên (Test Objective), Điều kiện tiên quyết, Đầu vào & Trạng thái (Inputs & State), và Kết quả mong đợi.

Xin hãy trình bày bằng Markdown rõ ràng để tôi có thể đưa trực tiếp vào báo cáo.

- **AI Output:**

Dưới đây là phần trình bày chi tiết cho kỹ thuật Boundary Value Analysis (BVA) của chức năng FR-08: Thanh toán (Checkout), được thiết kế đặc biệt để bắt các lỗi logic và tràn bộ nhớ do không giới hạn input. Bạn có thể copy toàn bộ đoạn Markdown này dán trực tiếp vào file Main_Report.md. Boundary Value Analysis (BVA) Bước 1: Phân tích các giá trị biên (Step-by-step Explanation) Dựa trên đặc tả hệ thống và việc giao diện UI không chặn độ dài của trường "Số lượng" (N), tôi tập trung phân tích 2 ranh giới cực độ có khả năng gây lỗi (Crash hoặc tạo đơn hàng ảo) tại API Checkout. Do hệ thống không quản lý tồn kho, mọi số lượng $N > 0$ đều được coi là hợp lệ về mặt logic nghiệp vụ, đẩy toàn bộ rủi ro về phía xử lý kiểu dữ liệu của Backend. 1. Ranh giới giới hạn tối thiểu của Giỏ hàng (Ngưỡng $N = 1$): Just below the boundary ($N = 0$): Kiểm tra khi người dùng cố tình nhập số lượng

là 0. Hệ thống phải từ chối thanh toán (tương đương giỏ hàng rỗng hoặc số lượng không hợp lệ). On the boundary (N = 1): Kiểm tra khi người dùng mua đúng 1 sản phẩm. Đây là mốc hợp lệ tối thiểu, hệ thống phải xử lý thành công. Just above the boundary (N = 2): Kiểm tra khi người dùng mua 2 sản phẩm, hệ thống phải xử lý tính toán tổng tiền chính xác và thanh toán thành công.

2. Ranh giới giới hạn lưu trữ tổng tiền (Integer Overflow - Ngưỡng Max Int 32-bit): Với kiểu dữ liệu số nguyên 32-bit có dấu, giá trị lớn nhất lưu trữ được là 2.147.483.647. Sản phẩm "iPhone 15 Pro Max" có giá 30.000.000 VNĐ. Ta phân tích quanh ranh giới gây tràn số:

Just below the boundary (N = 71): Tổng tiền là $71 * 30.000.000 = 2.130.000.000$ VNĐ (Nhằm sát dưới mức Max Int). Hệ thống vẫn phải xử lý thành công mà không bị sai lệch số liệu.

Just above the boundary / On Overflow (N = 72): Tổng tiền là $72 * 30.000.000 = 2.160.000.000$ VNĐ (Vượt mức Max Int). Việc truyền giá trị này có thể khiến Backend bị crash, lưu sai dữ liệu (biến thành số âm), hoặc báo lỗi Server (500). Kỳ vọng đúng là hệ thống sử dụng kiểu dữ liệu lớn hơn (như 64-bit/BigInt) hoặc có Validation bắt lỗi "Số tiền vượt quá giới hạn" một cách gọn gàng (Graceful error handling).

Bước 2: Thiết kế Test Case (BVA Testing)

Dưới đây là các Test Case chi tiết sử dụng mock data "iPhone 15 Pro Max" (30.000.000 VNĐ) để kiểm thử các giá trị biên:

Test Case ID	Tên (Test Objective)	Điều kiện tiền quyết (Precondition)	Đầu vào & Trạng thái (Inputs & State)	Kết quả mong đợi (Expected Result)
TC_FR-08_BVA_01	Kiểm tra biên dưới số lượng tối thiểu (N = 0)	User đã đăng nhập, giỏ hàng có "iPhone 15 Pro Max"	Sản phẩm: iPhone 15 Pro Max Số lượng (N): 0 Payload: total_amount = 0	Backend từ chối request, trả về lỗi 400 (Số lượng sản phẩm không hợp lệ hoặc giỏ hàng trống). Không tạo đơn hàng.
TC_FR-08_BVA_02	Kiểm tra tại biên số lượng tối thiểu (N = 1)	User đã đăng nhập, giỏ hàng có "iPhone 15 Pro Max"	Sản phẩm: iPhone 15 Pro Max Số lượng (N): 1 Payload: total_amount = 30.000.000	Thanh toán thành công, tạo đơn hàng với tổng tiền chính xác là 30.000.000 VNĐ.
TC_FR-08_BVA_03	Kiểm tra vượt biên số lượng tối thiểu (N = 2)	User đã đăng nhập, giỏ hàng có "iPhone 15 Pro Max"	Sản phẩm: iPhone 15 Pro Max Số lượng (N): 2 Payload: total_amount = 60.000.000	Thanh toán thành công, tạo đơn hàng với tổng tiền chính xác là 60.000.000 VNĐ.
TC_FR-08_BVA_04	Kiểm tra biên dưới giới hạn Integer 32-bit (N = 71)	User đã đăng nhập, giỏ hàng có "iPhone 15 Pro Max"	Sản phẩm: iPhone 15 Pro Max Số lượng (N): 71 Payload: total_amount = 2.130.000.000	Thanh toán thành công, tạo đơn hàng với tổng tiền chính xác là 2.130.000.000 VNĐ. Backend không bị lỗi số học.
TC_FR-08_BVA_05	Kiểm tra vượt biên giới hạn Integer 32-bit (N = 72) - Gây tràn số	User đã đăng nhập, giỏ hàng có "iPhone 15 Pro Max"	Sản phẩm: iPhone 15 Pro Max Số lượng (N): 72 Payload: total_amount = 2.160.000.000	Hệ thống xử lý an toàn: Có thể tạo đơn hàng thành công (nếu hỗ trợ BigInt/64-bit) HOẶC chặn request báo lỗi "Giá trị vượt giới hạn". Tuyệt đối không crash server (500) hoặc tạo đơn hàng với tổng tiền âm.

[Prompt 3]

- **Thời gian:** 16:00 09/06/2026
- **Tool:** Gemini 3.1 Pro
- **Prompt:**

tôi vẫn chưa log bug từ cái số 6 trở đi hãy cho tôi format từng cái 1 để tôi log lên nhé

- **AI Output:**

Chào bạn, tiến độ sẵn bug của bạn thực sự rất ấn tượng! Để chuẩn hóa toàn bộ quá trình đưa các lỗi này lên hệ thống theo đúng quy trình báo cáo, dưới đây là các template được thiết kế riêng cho từng GitHub Issue từ Bug 6 đến Bug 12. Mỗi template đã được đối chiếu kỹ lưỡng với đặc tả của FR-08 (Checkout) để đảm bảo phần "Kết quả mong đợi" hoàn toàn ăn khớp với yêu cầu của giảng viên. Bạn chỉ cần copy từng khối nội dung bên dưới và dán thẳng vào mục soạn thảo trên GitHub. Nhớ kéo thả hình ảnh tương ứng vào cuối mỗi bài nhé!

Issue 6: Lỗi không xóa giỏ hàngMarkdown**Tiêu đề:** [BUG] FR-08: Giỏ hàng không tự động xóa sau khi thanh toán thành công...

Issue 7: Lỗi hỏng thanh toán giá 0 VNĐMarkdown**Tiêu đề:** [BUG] FR-08: Lỗi hỏng bảo mật - Chấp nhận tạo đơn hàng với giá trị 0 VNĐ...

Issue 8: Lỗi hỏng thanh toán giá trị âmMarkdown**Tiêu đề:** [BUG] FR-08: Lỗi hỏng bảo mật - Chấp nhận tạo đơn hàng với giá trị âm...

Issue 9: Lỗi thanh toán đơn hàng rỗngMarkdown**Tiêu đề:** [BUG] FR-08: Khởi tạo đơn hàng thành công từ giỏ hàng rỗng (Empty Cart)...

Issue 10: Lỗi sập CORS khi thiếu TokenMarkdown**Tiêu đề:** [BUG] FR-08: API Checkout trả lỗi sập luồng CORS Policy thay vì mã 401 khi không có Token...

Issue 11: Lỗi sập CORS khi Token không hợp lệMarkdown**Tiêu đề:** [BUG] FR-08: API Checkout trả lỗi sập luồng CORS Policy khi JWT Token sai hoặc hết hạn...

Issue 12: Lỗi hỏng thanh toán số lượng 0Markdown**Tiêu đề:** [BUG] FR-08: Chấp nhận thanh toán sản phẩm với số lượng (Quantity) bằng 0...

FR-13: Trang chủ (Dashboard)

[Prompt 1]

- **Thời gian:** 14:00 23/06/2026
- **Tool:** Gemini 3.1 Pro
- **Prompt:**

Đóng vai trò là một Software Tester chuyên nghiệp. Tôi cần áp dụng kỹ thuật Domain Testing cho chức năng FR-13: Dashboard (Trang chủ Admin).

Dưới đây là đặc tả (business rules) của chức năng:

Hiển thị tổng doanh thu: Chỉ tính tổng total_amount của các đơn có status = 'delivered'.

Hiển thị tổng số đơn hàng: Đếm tổng số lượng tất cả các đơn hàng tồn tại trong hệ thống (không phân biệt trạng thái).

Giao diện (UI) hiện tại chỉ có 2 thẻ (cards) hiển thị 2 con số này, hoàn toàn không có bất kỳ bộ lọc thời gian (ngày/tháng/năm) nào.

LƯU Ý CỰC KỲ QUAN TRỌNG:

Chức năng này KHÔNG có input nhập liệu từ bàn phím hay nút bấm trên UI. "Input" ở đây chính là trạng thái dữ liệu (Data State) của các Đơn hàng lưu trong Database. KHÔNG tự vẽ ra các test case liên quan đến "chọn ngày", "lọc theo tháng", "chọn chi nhánh" hay "phân quyền". Chỉ tập trung vào việc hệ thống gom nhóm, đếm và tính tổng từ DB có đúng theo đặc tả hay không.

Nhiệm vụ của bạn gồm 2 phần, hãy trình bày hoàn toàn bằng Markdown để tôi đưa vào báo cáo:

Phần 1: Phân tích Domain (Step-by-step Explanation)

Hãy phân tích rõ các miền giá trị (Hợp lệ / Không hợp lệ / Loại trừ) cho các biến

trạng thái dữ liệu sau:

Biến số lượng đơn hàng (Database State: Trống vs Có dữ liệu).

Biến Trạng thái đơn hàng (status: 'delivered' vs các trạng thái khác như 'pending', 'cancelled'...).

Biến Giá trị đơn hàng (total_amount: dương, 0, âm - vì thực tế hệ thống đang có bug cho phép tạo đơn 0đ và số âm).

Phần 2: Thiết kế Bảng Test Case

Từ phân tích trên, hãy sinh ra một bộ Test Case chi tiết. Kết hợp các miền giá trị để tạo ra các kịch bản:

DB không có đơn hàng nào.

DB có đơn hàng nhưng chưa có đơn nào giao thành công.

DB có cả đơn giao thành công và đơn bị hủy/chờ xử lý (Mixed states).

DB có đơn hàng giao thành công nhưng giá trị là 0đ hoặc số âm.

Bảng gồm các cột: Test Case ID (Bắt đầu là TC_FR-13_01), Tên (Test Objective), Đầu vào & Trạng thái (Test Data), Hành động (Truy cập Dashboard), Kết quả mong đợi (Expected Result), Status.

Lưu ý rang toàn bộ kết quả trả về phải theo format markdown tương tự như các phần DomainTesting trước đó trong file Main report đính kèm.

• AI Output:

FR-13: Dashboard (Trang chủ)Domain TestingĐể thực hiện Domain Testing cho FR-13, do chức năng này không có input nhập liệu trực tiếp từ người dùng, tôi tiến hành phân tích "đầu vào" chính là trạng thái dữ liệu (Data State) của các Đơn hàng (Orders) được lưu trữ trong Database. Hệ thống EShop sẽ truy vấn, gom nhóm và tính toán dựa trên các dữ liệu này. Phân tích Miền giá trị (Domain Analysis): Dựa trên đặc tả hệ thống, chức năng Dashboard phụ thuộc vào 3 biến trạng thái dữ liệu cốt lõi:Biến trạng thái: Số lượng đơn hàng trong DB (Database State)Biến này quyết định việc hiển thị tổng số lượng đơn hàng. Miền hợp lệ (Valid Domain):V1 (Có dữ liệu): $N > 0$ (Hệ thống có chứa ít nhất 1 record đơn hàng).Miền biên vắng (Empty Domain):E1 (Trống): $N = 0$ (Database hoàn toàn trống, chưa có đơn hàng nào được tạo).Biến trạng thái: Trạng thái đơn hàng (status)Biến này quyết định điều kiện lọc để tính tổng doanh thu. Miền hợp lệ tính doanh thu (Revenue-counted Domain):V2: status = 'delivered' (Đơn hàng đã giao thành công, thỏa điều kiện cộng vào doanh thu). Miền loại trừ tính doanh thu (Excluded Domain):I1: status != 'delivered' (VD: pending, processing, cancelled...). Các đơn này chỉ được đếm vào "Tổng số đơn hàng", TUYỆT ĐỐI KHÔNG được cộng vào doanh thu. Biến dữ liệu: Giá trị đơn hàng (total_amount)Biến này là giá trị trực tiếp để cộng dồn vào doanh thu. Dựa trên thực tế hệ thống đã có lỗi hổng từ FR-08, ta có các miền sau: Miền hợp lệ thông thường (Standard Domain):V3: total_amount > 0 (Giá trị đơn hàng là số dương hợp logic).Miền ngoại lệ (Edge/Bug-derived Domains):E2: total_amount = 0 (Đơn hàng có giá 0 VNĐ, sinh ra từ lỗi hổng thiếu validation).E3: total_amount < 0 (Đơn hàng có giá trị âm, sinh ra từ lỗi hổng thiếu validation).Thiết kế Test Case (Domain Testing)Dựa trên các miền giá trị đã phân tích, dưới đây là tập hợp các Test Case đại diện nhằm đảm bảo bao phủ các kịch bản tính toán, đếm số lượng và xử lý các giá trị dị thường từ DB:Test Case IDTên (Test Objective)Đầu vào & Trạng thái (Test Data)Hành độngKết quả mong đợi (Expected Result)Kết quả thực tế (Actual Result)StatusTC_FR-13_01Kiểm tra Dashboard khi DB trống hoàn toàn (Phủ: E1)DB không có bất kỳ record đơn hàng nào.Truy cập trang Dashboard AdminTổng doanh thu hiển thị 0đ, tổng số đơn hàng hiển thị 0.Chưa thực hiệnN/ATC_FR-13_02Kiểm tra đếm số đơn nhưng không tính doanh thu khi chưa giao (Phủ: V1 + I1 + V3)DB có 3 đơn hàng dương tiền nhưng đều ở trạng thái pending, processing, cancelled.Truy cập

trang Dashboard AdminTổng doanh thu hiển thị 0đ, tổng số đơn hàng hiển thị 3.Chưa thực hiệnN/ATC_FR-13_03Kiểm tra tính doanh thu với trạng thái hỗn hợp (Happy Path) (Phủ: V1 + V2 + I1 + V3)DB có 4 đơn hàng:- Đơn 1: 1000k (delivered)- Đơn 2: 500k (delivered)- Đơn 3: 200k (pending)- Đơn 4: 100k (cancelled)Truy cập trang Dashboard AdminTổng doanh thu tính đúng 1.500.000đ (Chỉ cộng đơn 1 và 2), tổng số đơn hàng hiển thị 4.Chưa thực hiệnN/ATC_FR-13_04Kiểm tra tính doanh thu chứa đơn hàng 0 VNĐ (Phủ: V1 + V2 + E2)DB có 2 đơn hàng:- Đơn 1: 500k (delivered)- Đơn 2: 0đ (delivered)Truy cập trang Dashboard AdminTổng doanh thu tính đúng 500.000đ, tổng số đơn hàng hiển thị 2.Chưa thực hiệnN/ATC_FR-13_05Kiểm tra tính doanh thu chứa đơn hàng giá trị âm (Phủ: V1 + V2 + E3)DB có 2 đơn hàng:- Đơn 1: 1000k (delivered)- Đơn 2: -200k (delivered)Truy cập trang Dashboard AdminTổng doanh thu tính toán đúng giá trị đại số là 800.000đ (1000k - 200k), tổng số đơn hàng hiển thị 2.Chưa thực hiệnN/A

[Prompt 2]

- **Thời gian:** 21:30 24/06/2026
- **Tool:** Gemini 3.1 Pro
- **Prompt:**

Đóng vai trò là một Software Tester chuyên nghiệp. Chúng ta đã hoàn thành Domain Testing và phát hiện ra một lỗi nghiêm trọng: Hệ thống hiện tại đang tính sai tổng doanh thu bằng cách nhân đôi (x2) giá trị thực tế của các đơn hàng 'delivered' (Bug 13).

Bây giờ, tôi cần áp dụng kỹ thuật Boundary Value Analysis (BVA) cho chức năng FR-13: Dashboard (Trang chủ Admin).

Đặc tả (Business Rules):

Hiển thị tổng doanh thu: Chỉ tính tổng total_amount của các đơn có status = 'delivered'.

Hiển thị tổng số đơn hàng: Đếm tổng số lượng tất cả các đơn hàng.

Trạng thái hợp lệ chỉ gồm: pending, confirmed, shipping, delivered, canceled.

Tuyệt đối KHÔNG tự bịa thêm trạng thái processing hay các trạng thái khác.

Giao diện (UI) chỉ có 2 thẻ (cards) text, không có ô nhập liệu. Đầu vào (Input) chính là trạng thái dữ liệu trong Database.

Các ranh giới (Boundaries) cần phân tích:

Biên số lượng đơn hàng (N): N=0, N=1, và N cực lớn.

Biên tổng doanh thu (Integer Overflow / BigInt): Giới hạn của kiểu số nguyên 32-bit (Max Int = 2.147.483.647). Do hệ thống đang bị lỗi x2 doanh thu, nếu tổng tiền thực tế trong DB là 1.100.000.000đ (1.1 tỷ), khi tính toán hệ thống sẽ nhân đôi thành 2.200.000.000đ -> Vượt ranh giới Max Int 32-bit và có nguy cơ gây Crash / Tràn số (hiển thị số âm).

Biên hiển thị giao diện (UI Layout): Tổng tiền cực kỳ lớn (ví dụ: hàng trăm ngàn tỷ đồng - dài hơn 15 chữ số) để xem giao diện Card trên Dashboard có bị vỡ layout, tràn chữ ra ngoài thẻ hay không.

Nhiệm vụ của bạn:Bước 1 (Giải thích step-by-step): Viết một đoạn giải thích chi tiết cách phân tích các giá trị biên (min, max, just below, just above, on the boundary) cho các ranh giới tổng tiền và số lượng đề cập ở trên. Giải thích rõ việc lỗi x2 ảnh hưởng thế nào đến việc test tràn số.Bước 2 (Thiết kế Bảng Test Case): Thiết kế bảng Test Case BVA chi tiết.

Bảng gồm các cột: Test Case ID (Bắt đầu là TC_FR-13_BVA_01), Tên (Test Objective), Đầu vào & Trạng thái (Test Data trong DB), Kết quả mong đợi (Expected Result), Kết

quả thực tế (Actual Result), Status. (Để trống cột Kết quả thực tế và điền N/A cho cột Status để tôi tự test).

Yêu cầu trình bày hoàn toàn bằng định dạng Markdown để tôi copy trực tiếp vào main report.

• AI Output:

Dưới đây là phần trình bày chi tiết cho kỹ thuật Boundary Value Analysis (BVA) của chức năng FR-13: Dashboard (Trang chủ Admin). Bạn có thể copy toàn bộ nội dung Markdown này và dán trực tiếp vào phần BVA của FR-13 trong file

Main_Report.md.

Boundary Value Analysis (BVA)

Bước 1: Phân tích các giá trị biên (Step-by-step Explanation)

Khác với các form nhập liệu thông thường, giao diện Dashboard không có input trực tiếp. Ranh giới (boundary) ở đây nằm ở giới hạn kiểu dữ liệu của hệ thống Backend/Database và giới hạn hiển thị (layout) của Frontend. Đặc biệt, việc tồn tại Bug 13 (nhân đôi doanh thu) làm thay đổi hoàn toàn cách chúng ta tiếp cận ranh giới tràn bộ nhớ số nguyên (Integer Overflow). Dưới đây là phân tích chi tiết cho 3 ranh giới chính:

- Biên số lượng đơn hàng (N):**
 - Min boundary (N = 0):** Kiểm tra trạng thái rỗng khi Database chưa có dữ liệu. Tổng số đơn và tổng doanh thu đều phải là 0.
 - Just above min (N = 1):** Kiểm tra khi hệ thống có đúng 1 đơn hàng đầu tiên.
 - Max boundary (N cực lớn):** Kiểm tra khi số lượng đơn hàng lên tới hàng trăm ngàn hoặc hàng triệu (VD: N = 999.999) xem việc đếm (COUNT) có làm chậm hệ thống hay vỡ UI của thẻ hiển thị số lượng hay không.
- Biên lưu trữ Tổng doanh thu (Integer Overflow) & Tác động của Bug 13:**
 - Giới hạn tối đa của kiểu số nguyên 32-bit (Max Int có dấu)** là 2.147.483.647. Thông thường, ta sẽ test giá trị trong DB mập méo mốc 2.14 tỷ. Tuy nhiên, do Backend đang mắc Bug 13 (tự động x2 doanh thu), ngưỡng tràn số thực tế đối với dữ liệu đầu vào (DB) bị giảm đi một nửa (chỉ còn khoảng 1.073.741.823).
 - Just below boundary:** Tạo đơn hàng trong DB có giá trị 1.070.000.000đ. Khi tính toán, Backend x2 thành 2.140.000.000đ (Vẫn nằm trong giới hạn an toàn của 32-bit). Hệ thống không được crash.
 - Just above boundary (Vượt biên):** Tạo đơn hàng trong DB có giá trị 1.100.000.000đ (1.1 tỷ). Backend sẽ x2 thành 2.200.000.000đ. Mức này vượt qua mốc Max Int 32-bit. Ta cần test xem Backend có crash (lỗi 500), hay trả về một số âm dị thường do tràn bit, hay hệ thống đã phòng ngừa bằng cách dùng kiểu dữ liệu BigInt/Long (64-bit).
- Biên hiển thị giao diện (UI Layout):**
 - Tràn UI:** Cố tình tạo một tổng doanh thu cực kỳ lớn, dài hơn 15-20 chữ số (Ví dụ: 100.000.000.000.000đ - 100 ngàn tỷ) để xem Frontend xử lý hiển thị thế nào. Text có bị tràn ra ngoài thẻ (Card) làm hỏng bố cục CSS, hay được tự động rút gọn (dùng dấu ... hoặc format 100T) một cách gọn gàng.

Bước 2: Thiết kế Test Case (BVA Testing)

Dưới đây là các Test Case chi tiết để kiểm thử các giá trị biên đã phân tích đối với chức năng Dashboard:

Test Case ID	Tên (Test Objective)	Đầu vào & Trạng thái (Test Data trong DB)	Kết quả mong đợi (Expected Result)	Kết quả thực tế (Actual Result)	Status
TC_FR-13_BVA_01	Kiểm tra biên dưới số lượng đơn (N = 0)	DB hoàn toàn trống (0 đơn hàng)	Dashboard hiển thị số lượng: 0. Doanh thu: 0đ.	N/A	TC_FR-13_BVA_01
TC_FR-13_BVA_02	Kiểm tra sát biên dưới số lượng đơn (N = 1)	DB có 1 đơn hàng delivered (Giá: 500.000đ)	Dashboard hiển thị số lượng: 1. (Doanh thu x2 thành 1.000.000đ do Bug 13)	N/A	TC_FR-13_BVA_02
TC_FR-13_BVA_03	Kiểm tra biên cận dưới giới hạn Max Int 32-bit (Test an toàn)	DB có 1 đơn hàng delivered giá trị: 1.070.000.000đ	Hệ thống nhân đôi thành 2.140.000.000đ (Chưa vượt Max Int). Dashboard hiển thị bình thường, Backend không bị lỗi số học hay crash.	N/A	TC_FR-13_BVA_03
TC_FR-13_BVA_04	Kiểm tra vượt biên Max Int 32-bit (Gây tràn số do Bug 13)	DB có 1 đơn hàng delivered giá trị: 1.100.000.000đ (1.1 Tỷ)	Mức x2 là 2.200.000.000đ vượt mốc 32-bit. Kỳ vọng hệ thống dùng kiểu BigInt/Long để hiển thị chính xác số này. Tuyệt đối không được văng lỗi 500 Internal Server Error hay hiển thị doanh thu âm.	N/A	TC_FR-13_BVA_04
TC_FR-13_BVA_05	Kiểm tra giới hạn hiển thị UI (Vỡ				

Layout)DB có đơn hàng delivered tổng giá trị siêu lớn: 100.000.000.000.000đGiao diện Dashboard có dẫn hợp lý hoặc cắt bớt chuỗi text. Chữ số không được phép tràn ra ngoài khung Card hiển thị hay đề lên các thành phần khác.N/ATC_FR-13_BVA_06Kiểm tra hiển thị UI với số lượng đơn hàng (N) cực lớnDB chứa 999.999 đơn hàng ở trạng thái canceled và delivered.Số đếm đơn hàng hiển thị 999.999 vừa vặn trong Card. Hệ thống không bị timeout (quá thời gian truy vấn) khi đếm số lượng bản ghi lớn.N/A

Agent Skill

[Prompt 1]

- **Thời gian:** 14:30 29/06/2026
- **Tool:** Gemini 3.1 Pro
- **Prompt:**

tôi đã làm xong 3 trong số 4 fr, và đề có yêu cầu như sau:

Agent Skill: You are encouraged to build Agent Skills that apply the domain testing and boundary value analysis techniques, so that they can be reused on additional features in future testing tasks. Submit the skills together with demonstration videos YouTube links that show, end to end, how you used the skills on a complete feature.

Tôi dự tính sẽ sử dụng chính fr-20: "FR-20: Giỏ hàng (Shopping Cart) trên Mobile: Kiểm thử các thao tác vuốt, chạm, và cập nhật số lượng (Boundary Value cho số lượng min/max) trực tiếp trên màn hình cảm ứng." để làm test cho phần skill này, tuy nhiên trước tiên là tôi cần xây dựng skill đã, mà tôi đã không biết nên làm như nào. Bạn tôi có gợi ý tôi là tạo skill ngay trong vs code nhưng tôi không biết làm sao. Bạn có thể hỗ trợ tôi không? Đây là file đề bài (2026....) và tất cả các file có liên quan trong bài tập lần này. trong đó quan trọng nhất là file Mai Report chứa các fr. Tôi đã làm sẵn 3 fr với format đầy đủ. Bạn hãy dùng nó để tham khảo cho skill agent nhé.

- **AI Output:**

Bước 1: Khái niệm về Agent Skill trong VS CodeTrong VS Code, nếu bạn dùng các AI extension (như GitHub Copilot Chat, Cline, RooCode, hoặc Cursor), bạn có thể tạo một file .md chứa toàn bộ chỉ thị. Khi chat với AI, bạn chỉ cần yêu cầu: "Hãy đọc file Hướng dẫn này và áp dụng nó để phân tích đặc tả chức năng sau...". Đó chính là cách Agent Skill hoạt động!Bước 2: Tạo file Agent Skill (Khung tư duy của Tester)Trong thư mục dự án trên VS Code, bạn hãy tạo một file mới đặt tên là Eshop_QA_Agent_Skill.md. Sau đó, copy toàn bộ nội dung dưới đây dán vào file đó. (Nội dung này được tôi đúc kết từ chính file Main_Report.md của bạn để AI bắt chước chính xác 100% phong cách của bạn).Markdown# ESHOP QA AGENT SKILL - SYSTEM INSTRUCTIONS

****Role:**** Bạn là một Senior Software Tester. Nhiệm vụ của bạn là áp dụng kỹ thuật Domain Testing và Boundary Value Analysis (BVA) để thiết kế Test Case cho hệ thống EShop.

****Context & Constraints:****

1. Trình bày 100% bằng tiếng Việt và sử dụng định dạng Markdown.

2. KHÔNG tự bịa đặt (hallucinate) dữ liệu, trạng thái, hoặc tính năng không có trong đặc tả. Hệ thống KHÔNG có chức năng quản lý tồn kho (stock).

3. Các bảng Test Case PHẢI luôn có cột: "Kết quả thực tế (Actual Result)" (để trống) và "Status" (điền N/A).

****Workflow (Quy trình thực thi):****

Khi User cung cấp cho bạn [Tên chức năng] và [Đặc tả], bạn phải thực hiện tuần tự 4 bước sau, gom vào một câu trả lời duy nhất:

BƯỚC 1: DOMAIN TESTING ANALYSIS

- Phân tích chi tiết các biến đầu vào (Inputs, bao gồm cả thao tác người dùng như vuốt/chạm nếu là Mobile) và biến trạng thái (State).
- Xác định rõ các Miền hợp lệ (Valid Domains) và Miền không hợp lệ (Invalid/Edge Domains).
- Dựa vào phân tích, sinh ra bảng "Thiết kế Test Case (Domain Testing)".
- Format bảng: `| Test Case ID | Tên (Test Objective) | Đầu vào & Trạng thái (Test Data) | Kết quả mong đợi (Expected Result) | Kết quả thực tế (Actual Result) | Status |`
- Cột Test Case ID có format: `TC\_[ID Chức Năng]\_[Số thứ tự]`

BƯỚC 2: BOUNDARY VALUE ANALYSIS (BVA)

- Phân tích các ranh giới (Boundaries) từ đặc tả (ví dụ: Số lượng Min, Max, hoặc giới hạn kích thước màn hình, thao tác vật lý).
- Giải thích rõ cách chọn giá trị: Min, Min+1, Max-1, Max.
- Sinh ra bảng "Thiết kế Test Case (BVA Testing)" với format giống hệt bảng Domain Testing.
- Cột Test Case ID có format: `TC\_[ID Chức Năng]\_BVA\_[Số thứ tự]`

BƯỚC 3: AI GAP ANALYSIS (Khoảng trống AI)

- Bạn (AI) hãy tự nhận xét xem với đặc tả này, có trường hợp nào bạn dễ bị bỏ sót không (ví dụ: thiếu UI testing, thiếu phần cứng, hoặc giới hạn về việc không có thiết bị mobile thật để test vuốt/chạm).

BƯỚC 4: BUG REPORT TEMPLATE

- Sinh sẵn 1 template Bug Report theo chuẩn GitHub Issue để User có thể dùng ngay nếu phát hiện lỗi trong quá trình thực thi test.
- Bao gồm: Tiêu đề, Mô tả lỗi, Bước tái hiện, Kết quả mong đợi, Kết quả thực tế, Test Case liên quan, Ảnh minh chứng.

Bước 3: Kích hoạt Agent Skill để xử lý FR-20. Bây giờ, Agent Skill đã sẵn sàng. Bạn mở bảng Chat của AI trong VS Code (ví dụ: Copilot Chat) và đưa ra câu Prompt kích hoạt như sau: Prompt kích hoạt trong VS Code: "Hãy đóng vai trò là Eshop QA Agent. Vui lòng đọc file @Eshop_QA_Agent_Skill.md để hiểu quy trình làm việc và format đầu ra. Sau đó, áp dụng toàn bộ kỹ năng đó để xử lý chức năng sau: [Tên chức năng]: FR-20: Giỏ hàng (Shopping Cart) trên Mobile [Đặc tả]: Môi trường: Ứng dụng Mobile (Màn hình cảm ứng). Tính năng: Kiểm thử các thao tác vuốt (swipe) để xóa sản phẩm, chạm (tap) để chọn. Cập nhật số lượng: Cập nhật số lượng trực tiếp trên giao diện. (Gợi ý cho BVA: Số lượng tối thiểu là 1, số lượng tối đa có thể bị chặn bởi 2 chữ số là 99). Giao diện tự động tính lại tổng tiền khi số lượng thay đổi. Xóa sản phẩm: Khi số lượng giảm xuống 0 hoặc vuốt để xóa, sản phẩm biến mất khỏi giỏ. Hãy thực thi cả 4 bước được quy định trong Skill nhé." Bước 4: Chạy Test và Quay Video (Lấy điểm Agent Skill) Đây là kịch bản hoàn hảo để bạn quay video màn hình: Mở đầu video: Bật VS Code lên, mở file Eshop_QA_Agent_Skill.md và giới thiệu ngắn gọn: "Đây là Agent Skill tôi xây dựng để tự động hóa tư duy Domain Testing và BVA, chống AI bịa đặt dữ liệu và ép nó trả về đúng format báo cáo". Kích hoạt: Mở cửa sổ Chat của AI, copy câu "Prompt kích hoạt" ở Bước 3 dán vào và nhấn Enter. Show kết quả: AI sẽ chạy một mạch tuôn ra toàn bộ: Phân tích Domain (có thao tác vuốt/chạm), BVA (với min = 1, max = 99, 100), AI Gap và Template Bug. Cuộn màn hình để GV thấy kết quả rất chuyên nghiệp. Thực thi thực tế: Chuyển sang trình duyệt (bật chế độ Mobile view - F12) hoặc máy ảo Emulator. Lấy thử 1 test case

vuốt để xóa, hoặc nhập số lượng 0 xem nó có xóa sản phẩm thật không. Log Bug: Nếu có lỗi (chắc chắn FR này có lỗi), copy cái Template Bug AI vừa sinh ra, điền thông tin lỗi bạn vừa bắt được vào. Kết thúc video. Cách làm này không chỉ giúp bạn hoàn thành FR-20 cực kỳ nhàn nhã, mà còn chứng minh được bạn đã hiểu và làm chủ hoàn toàn công cụ AI để tạo ra một quy trình làm việc chuẩn mực! Bạn hãy thử tạo file và chạy lệnh trong VS Code xem AI trả về kết quả mượt mà như thế nào nhé.

FR-20: Giỏ hàng trên Mobile (Mobile Cart)

[Prompt 1] (Agent Skill Execution)

- **Thời gian:** 29/06/2026 14:30
- **Tool:** Eshop QA Agent Skill
- **Prompt:**

****System Core Blueprint (Quy tắc hệ thống):****

(Trích xuất tóm tắt nhiệm vụ của Agent: Áp dụng Domain Testing, BVA, tuân thủ định dạng Markdown bảng biểu, không hallucinate, xuất 3 file.)

****Input Feature Specification Used (Đặc tả đầu vào):****

FR-20: Giỏ hàng (Shopping Cart) trên Mobile.

- Môi trường: Ứng dụng Mobile (Màn hình cảm ứng).
- Tính năng: Kiểm thử các thao tác vuốt (swipe) để xóa sản phẩm, chạm (tap) để chọn.
- Cập nhật số lượng: Cập nhật số lượng trực tiếp trên giao diện.
- Gợi ý cho BVA: Số lượng tối thiểu là 1, số lượng tối đa không giới hạn.
- Giao diện tự động tính lại tổng tiền khi số lượng thay đổi.
- Xóa sản phẩm: Khi số lượng giảm xuống 0 hoặc vuốt để xóa, sản phẩm biến mất khỏi giỏ.

- **AI Output:**

Tóm tắt ngắn gọn: Agent đã phân tích thành công các miền hợp lệ/không hợp lệ cho thao tác tap và swipe, thiết kế bộ test Domain và BVA bám sát biên số lượng 1/0, đồng thời tạo template Bug Report theo chuẩn GitHub Issue. Lưu ý quan trọng: do đặc tả không nêu ngưỡng tối đa số lượng, phần BVA chỉ kiểm thử biên dưới và các trạng thái chuyển về 0 để tránh tự đặt giới hạn không có trong yêu cầu.

AI Audit Report

Sản phẩm 1 (Artifact 1): Phân tích Domain và BVA cho chức năng Đăng nhập (FR-02) và Thanh toán (FR-08)

- **(1) Prompt + Tool:**

- **Tool:** Gemini 3.1 Pro
- **Thời gian:** 09/06/2026 - 16/06/2026
- **Prompt:** Các yêu cầu phân tích miền giá trị và BVA dựa trên đặc tả hệ thống cho FR-02 và FR-08.
- **(2) AI output:** AI sinh ra bảng test case chi tiết nhưng sử dụng sai thông tin tài khoản test (dùng user@example.com thay vì test@eshop.com) và thiếu các cột kết quả thực thi. Ở FR-08, AI tự động tạo các kịch bản kiểm tra tồn kho (stock) và bỏ quên luồng thao tác trên giao diện của Khách vắng lai (Guest).
- **(3) Kết luận:** CHƯA HOÀN THIỆN (INCOMPLETE) & KHÔNG HỢP LỆ (INVALID)
- **(4) Lý do:** AI bị "ảo giác" (hallucination) khi tự áp đặt cơ chế tồn kho vốn không có trong hệ thống SUT. Đồng thời, AI thiếu ngữ cảnh về trải nghiệm người dùng (UX) trên trình duyệt nên không thiết kế kịch bản UI cho trường hợp thao tác mà thiếu Token.
- **(5) Chỉnh sửa:** Sửa lại toàn bộ dữ liệu test data thành test@eshop.com và iPhone 15 Pro Max. Đánh dấu bỏ qua (N/A) các test case liên quan đến tồn kho và tự bổ sung kịch bản kiểm tra chặn thanh toán UI khi chưa đăng nhập.

Sản phẩm 2 (Artifact 2): Phân tích Domain và BVA cho chức năng Dashboard (FR-13)

- **(1) Prompt + Tool:**
 - **Tool:** Gemini 3.1 Pro
 - **Thời gian:** 23/06/2026 - 24/06/2026
 - **Prompt:** Yêu cầu thiết kế Bảng Test Case phân tích số lượng đơn hàng và tổng doanh thu, lưu ý bám sát ranh giới khi có lỗi nhân đôi doanh thu (Bug 13).
- **(2) AI output:** AI phân tích đúng ranh giới của dữ liệu nhưng lại tự động thêm trạng thái processing vào kịch bản kiểm thử và tự bịa ra các con số tiền tệ ngẫu nhiên như 1.000.000đ, 200.000đ.
- **(3) Kết luận:** CHƯA HOÀN THIỆN (INCOMPLETE) & KHÔNG HỢP LỆ (INVALID)
- **(4) Lý do:** AI áp dụng kiến thức chung về các hệ thống E-commerce thực tế thay vì tuân thủ chặt chẽ ranh giới vòng đời trạng thái (State Machine) của hệ thống SUT hiện tại. Việc sinh dữ liệu (Arbitrary Data Generation) không bám vào mock data khiến bài test thiếu tính thực tế.
- **(5) Chỉnh sửa:** Loại bỏ hoàn toàn trạng thái processing do không hợp lệ. Cập nhật lại toàn bộ test data bằng các con số tiền tệ tính toán chính xác để làm nổi bật tác động của Bug 13 lên nguy cơ tràn số nguyên (Max Int 32-bit).

Sản phẩm 3 (Artifact 3): Thiết lập và thực thi EShop QA Agent Skill cho Giỏ hàng (FR-20)

- **(1) Prompt + Tool:**
 - **Tool:** EShop QA Agent Skill (chạy qua Copilot AI Chatbox Auto Model)
 - **Thời gian:** 29/06/2026
 - **Prompt:** Kích hoạt Agent Skill với System Blueprint ép buộc AI dùng đúng định dạng Markdown, không hallucinate tồn kho và chạy luồng BVA cho FR-20.
- **(2) AI output:** Tạo ra tài liệu có định dạng cực kỳ chuẩn xác và tạo sẵn Bug Report Template theo chuẩn. Tuy nhiên, về mặt logic, AI vẫn mặc định chức năng giảm số lượng về 0 sẽ tự xóa sản phẩm khỏi giỏ, bỏ qua việc yêu cầu Dialog xác nhận, và không có kịch bản test trạng thái Đăng xuất.
- **(3) Kết luận:** CHƯA HOÀN THIỆN (INCOMPLETE) & KHÔNG HỢP LỆ (INVALID)
- **(4) Lý do:** Mặc dù Agent Skill khắc phục hoàn toàn vấn đề định dạng và cấu trúc bảng, bản chất mô hình LLM vẫn phân tích cục bộ trong giới hạn một UI nhỏ bé. Sự thiếu góc nhìn tổng thể về Client-Server đã dẫn đến lỗi hổng bỏ sót việc Session Management.

- **(5) Chính sửa:** Tôi trực tiếp thiết kế thêm Test Case 07 để kiểm thử quản lý phiên, qua đó bắt được Bug 19 (Rò rỉ dữ liệu phiên). Cấu trúc lại BVA để kiểm chứng cơ chế chặn số lượng ở mức tối thiểu 1 và cập nhật các Bug 16, 17, 18 liên quan đến UI thực tế vào tài liệu.

Tổng kết và Kết luận

1. Tỷ lệ chính xác của AI (AI Accuracy Ratio): Dựa trên các Artifacts trong quá trình thực thi HW02:

- **VALID:** 0% (Chưa có sản phẩm đầu ra nào hoàn hảo 100% để copy-paste trực tiếp mà không sửa đổi).
- **INVALID:** 30% (Một số chỗ bị sai lệch logic, tự sáng tạo trạng thái và tự thiết kế kịch bản quản lý kho không có thật).
- **INCOMPLETE:** ~70% (Xây dựng form đúng nhưng thiếu kịch bản biên quan trọng như Race Condition, kiểm soát Session, hoặc xử lý thiếu vắng Token trên UI).

2. Kết luận: Qua quá trình hợp tác và kiểm toán công cụ Gemini 3.1 Pro và Copilot, tôi rút ra nguyên tắc sử dụng AI trong công việc QA/QC như sau:

- **KHI NÀO NÊN DÙNG AI:** Sử dụng AI hiệu quả nhất thông qua việc thiết lập các Agent Skill (System Instructions) để tạo bộ khung (templates) kiểm thử, chuẩn hóa bảng biểu báo cáo, và tư duy nhanh các kịch bản luồng chuẩn (Happy Path).
- **KHI NÀO KHÔNG NÊN DÙNG AI:** Không tin tưởng hoàn toàn vào khả năng phân tích các tính năng có tính liên kết chặt chẽ tới biến ẩn (ví dụ: Session, Authentication) của AI. Tuyệt đối không để AI tự quyết định logic hệ thống (như tự thêm tồn kho) mà không đối chiếu tài liệu đặc tả. Sự kiểm soát thủ công (Human-in-the-loop) là bắt buộc để tìm ra những lỗi ẩn sâu như Race Condition hay Memory Leak.